Mythili.S - 241801178

S.No.	Date	Title	Teacher's sign
1.	24/7/26	Network commands used in LINUX.	
2.	26/7/26	CISCO Packet Tracer	
3.	6/8/26	Study of different types of Network cables.	
4.	6/08/2026	Study of local area network using a switch and Ethernet cables	
5.	06/08/2026	Packet capture using wire shark.	
6.	8/8/2026	Error detection and correction using hamming code.	
7.	13/8/2026	Flow - control of data link layer using Sliding window Protocol.	
8.	24/8/2026	Implementation of Subnetting in cisco packet tracer.	
9.	24/8/2026	a) Internet working using routers.	
	30/8/2026	b) Wireless Router, DHCP server and Internet cloud.	
10.	30/8/2026	Routing at Network layer.	
12.	22/9/2026	Implement Echo client server using TCP/IP Sockets.	
13.	26/9/2026	PING Program.	

14.	30/9/2026	Network packet sniffer using raw sockets.	
15.	5/10/2026	Types of weblogs using webalizer log analysis.	

Completed

EXP: NO: 1
DATE: 24/7/26

Network Commands used In Linux

Aim:

To study various network commands used in Linux and windows.

commands:

aap -a → Shows IP address of the computer along with the IP and MAC address.

hostname → Displays name of the computer.

ipconfig → Displays configuration in the PC.

nbtstat -a → Helps to solve problems with Notations NetBios.

nslookup → Displays detail DNS

Pathping → Traces the router to the destination address then launches a 25 second test of each router the way.

Ping → Best way to test connectivity blw two nodes.

#Ping hostname
#Ping ipaddress
#Ping

Result:

Thus the network commands has been studied and executed successfully.

```
If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.

Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*                .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
              destination^      ^mask      ^gateway     metric^    ^
                                                        Interface^

    If IF is not given, it tries to find the best interface for a given
    gateway.
    > route ADD 3ffe::/32 3ffe::1

    > route CHANGE 157.0.0.0 MASK 255.0.0.0 157.55.80.5 METRIC 2 IF 2

    CHANGE is used to modify gateway and/or metric only.

    > route DELETE 157.0.0.0
    > route DELETE 3ffe::/32
```

```
C:\Users\Tcs>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

RemoteName   Remote host machine name.
IP address   Dotted decimal representation of the IP address.
interval     Redisplays selected statistics, pausing interval seconds
             between each display. Press Ctrl+C to stop redisplaying
             statistics.
```

EXP: NO: 1
DATE: 24/7/26

Aim:

TO study

indows.

mands:

-a → Sho

IP a

ame → Di

ig → Disp

at -a →

ackup → D

Pathping → TC

la

Best w

#Ping

#Ping

#Ping

ues

Execut

```
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Tcs>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a            Displays current ARP entries by interrogating the current
                protocol data.  If inet_addr is specified, the IP and Physical
                addresses for only the specified computer are displayed.  If
                more than one network interface uses ARP, entries for each ARP
                table are displayed.
  -g            Same as -a.
  -v            Displays current ARP entries in verbose mode.  All invalid
                entries and entries on the loop-back interface will be shown.
  inet_addr     Specifies an internet address.
  -N if_addr    Displays the ARP entries for the network interface specified
                by if_addr.
  -d            Deletes the host specified by inet_addr. inet_addr may be
                wildcarded with * to delete all hosts.
  -s            Adds the host and associates the Internet address inet_addr
                with the Physical address eth_addr.  The Physical address is
                given as 6 hexadecimal bytes separated by hyphens. The entry
                is permanent.
  eth_addr      Specifies a physical address.
  if_addr       If present, this specifies the Internet address of the
                interface whose address translation table should be modified.
                If not present, the first applicable interface will be used.
Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.
```

```
C:\Users\Tcs>ne

Active Connecti

  Proto  Local
  TCP    127.0.
  TCP    127.0.
  TCP    127.0.
  TCP    127.0.
  TCP    172.16
  TCP    172.16
  TCP    172.16
  TCP    172.16
  TCP    172.16
  TCP    172.16
  TCP    172.16
  TCP    172.16
  TCP    [fe80:
  TCP    [fe80:
```

```
C:\Users\Tcs>hostname
DESKTOP-G0LVE4J

C:\Users\Tcs>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::d8a1:e21b:7bd7:849c%11
   IPv4 Address. . . . . . . . . . . : 172.16.10.80
   Subnet Mask . . . . . . . . . . . : 255.255.252.0
   Default Gateway . . . . . . . . . : 172.16.8.1
```

```
C:\Users\Tcs>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    127.0.0.1:49682        DESKTOP-G0LVE4J:49683  ESTABLIS
  TCP    127.0.0.1:49683        DESKTOP-G0LVE4J:49682  ESTABLIS
  TCP    127.0.0.1:49684        DESKTOP-G0LVE4J:49685  ESTABLIS
  TCP    127.0.0.1:49685        DESKTOP-G0LVE4J:49684  ESTABLIS
  TCP    172.16.10.80:53081     4.213.25.240:https     ESTABLIS
  TCP    172.16.10.80:53471     4.213.25.240:https     ESTABLIS
  TCP    172.16.10.80:53476     a23-202-229-22:https   ESTABLIS
  TCP    172.16.10.80:53477     57.155.48.216:https    ESTABLIS
  TCP    172.16.10.80:53478     204.79.197.222:https   ESTABLIS
  TCP    172.16.10.80:53480     150.171.110.84:https   ESTABLIS
  TCP    172.16.10.80:53481     172.178.232.44:https   ESTABLIS
  TCP    172.16.10.80:53482     172.16.7.152:ms-do     SYN_SENT
  TCP    [fe80::d8a1:e21b:7bd7:849c%11]:1521  DESKTOP-G0LVE4J:4
  TCP    [fe80::d8a1:e21b:7bd7:849c%11]:49680  DESKTOP-G0LVE4J:
```

EXP: NO: 3
DATE: 26/7/26

CISCO PACKET TRACER

Aim:

TO study the Packet tracer tool Installation and user Interface overview.

C)TO understand environment of CISCO PACKET TRACER to design simple network.

Introduction:

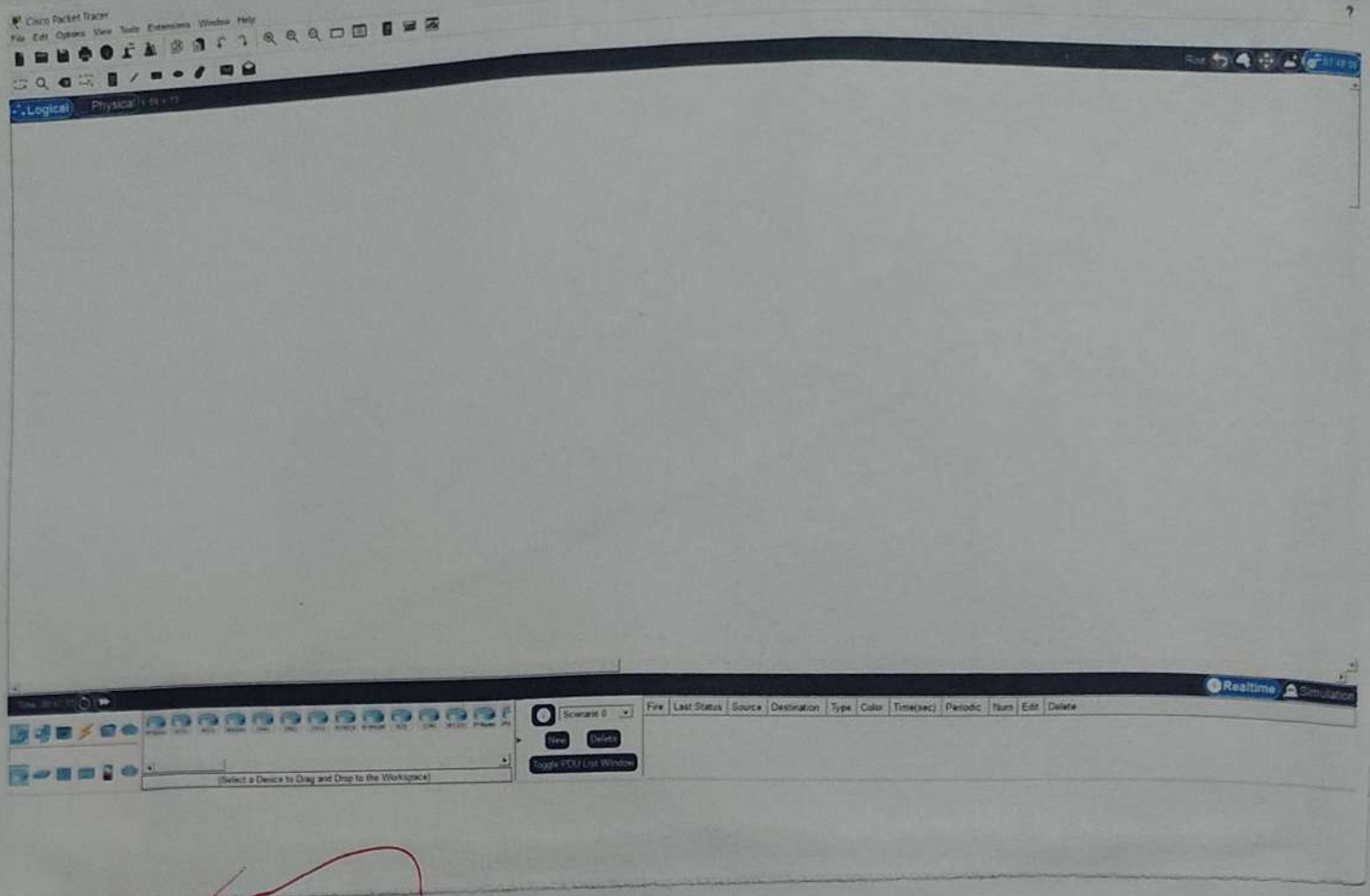
A Simulator, as the name suggests, simulates network devices and its environment. Packet Tracer is an exciting network design, Simulation and modeling tool.

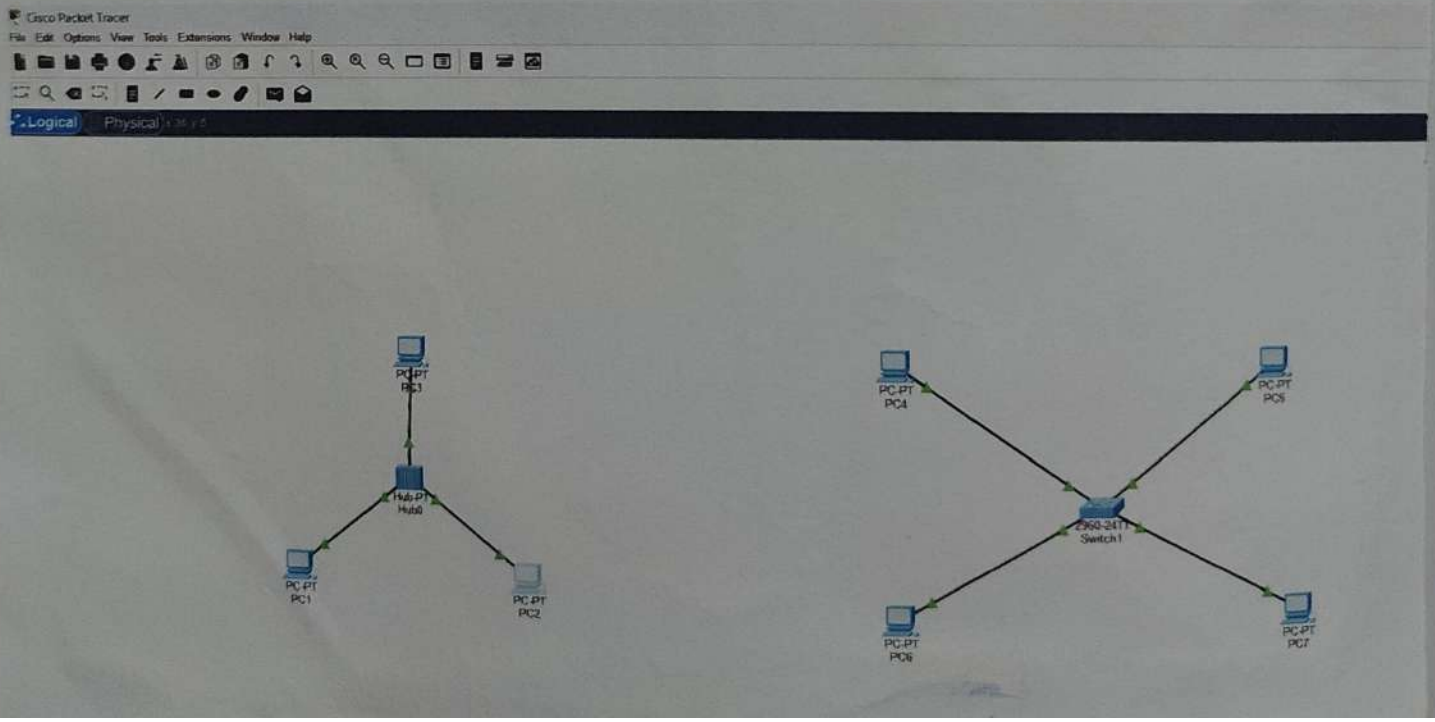
1. It allows you to model complex systems without the need for dedicated equipment.

2. It helps you to practice your network configuration and troubleshooting skills via computer or an Android or iOS based mobile device.

3. It is available for both the Linux and windows desktop environments.

4. Protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware.


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical
Physical
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
(Select a Device to Drag and Drop to the Workspace)


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical
Physical
PC-PT
PC3
Hub-PT
Hub0
PC-PT
PC1
PC-PT
PC2
PC-PT
PC4
PC-PT
PC5
2960-24TT
Switch1
PC-PT
PC6
PC-PT
PC7

Installing Packet Tracer:

To download Packet Tracer, go to https://www.netacad.com and login with your cisco Networking Academy credentials; then, click on the Packet Tracer graphic and download the package appropriate for your operating system.

Windows:

User Interface overview:

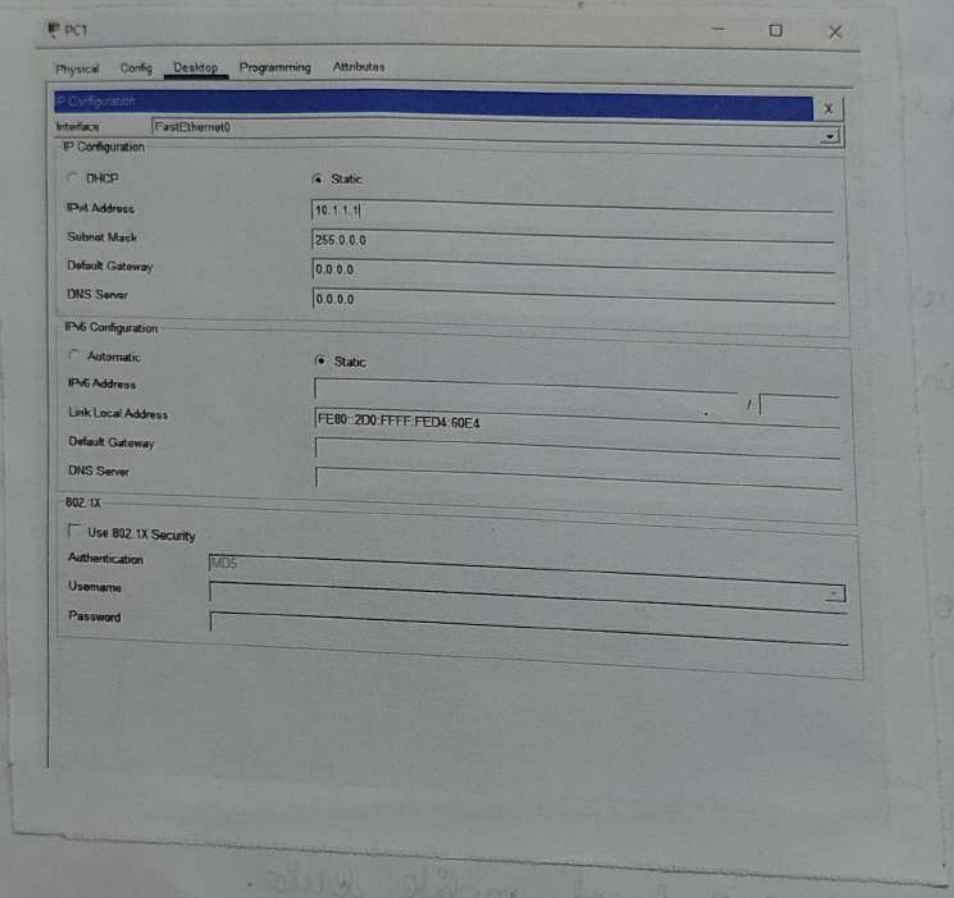
The layout of Packet Tracer is divided into several components. The components of the Packet Tracer interface are as follows.

1. Menu bar - This is a common menu found in all software applications; it is used to open, save, print, change preferences, and so on.

2. Main toolbar - This bar provides shortcut icons to menu options that are commonly accessed, such as open, save, zoom, undo, and redo, and on the right-hand side is an icon for entering network information for the current network.

3. Logical/Physical workspace tabs - These tabs allow you to toggle between the logical and physical work areas.

4. Workspace - This is the area where topologies are created and simulations are displayed.


PC1
Physical
Config
Desktop
Programming
Attributes
IP Configuration
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
10.1.1.1
Subnet Mask
255.0.0.0
Default Gateway
0.0.0.0
DNS Server
0.0.0.0
IPv6 Configuration
Automatic
Static
IPv6 Address
Link Local Address
FE80::2D0:FFFF:FED4:60E4
Default Gateway
DNS Server
802.1X
Use 802.1X Security
Authentication
MD5
Username
Password

5. Common tools bar - this toolbar provides controls for manipulating topologies, such as Select, move layout, place note, delete, inspect, resize shape, and add simple/complex PDU.

6. Real-time/simulation tabs - these tabs are used to toggle between the real and simulation modes. Buttons are also provided to control the time, and to capture the packets.

7. Network component box - this component contains all of the network and end devices available with packet Tracer. and is further divided into two areas: when a device category is selected, this selection box displays the different device models within that category.

8. User-created packet box - users can create highly-customized packets to test their topology from this area, and the results are displayed as a list.

d) Analyse the behaviour of network devices using CISCO PACKET TRACER simulator.

1. From the network box, click and drag-and-drop the below components:

a. 4 Generic PCs and One HUB

b. 4 Generic PCs and one Switch.

PC2

Physical Config Desktop Programming Attributes

IP Configuration

Interface FastEthernet0

IP Configuration

DHCP Static

IPv4 Address 10.1.1.2

Subnet Mask 255.0.0.0

Default Gateway 0.0.0.0

DNS Server 0.0.0.0

IPv6 Configuration

Automatic Static

IPv6 Address /

Link Local Address FE80::250:FFF:FEE9:B2C6

Default Gateway

DNS Server

802.1X

Use 802.1X Security

Authentication MD5

Username

Password

2. click on connections :

a. click on copper straight-through cable.

b. Select one of the PC and connect it to HUB using the cable. the link LED should glow. in green, indicating that the link is up. Similarly connect remaining 3PCs to the HUB.

c. Similarly connect 4 PCs to the switch using copper straight-through cable.

3. click on the PCs connected to hub, go to the Desktop tab, click on IP configuration, and enter an IP address and subnet mask. Here, the default gateway and DNS server information is not allowed as there are only two end devices in the network.

click on the PDU from the common tool bar.

a) Drag and drop it on the PC and then drop it on another PC connected to the HUB.

4. observe the flow of (PDU) from source of PC to destination PC by selecting the Realtime mode of simulation.

5. Repeat step 3 to step 5 for PCs to Connect to switch.

6. observe the HUB and switch are forward the PDU & write observation & conclusion.

Result:

Thus the study of packet tracer tool installation and user interface overview was successfully implemented.

EXP NO: 2
DATE: 6/8/26

STUDY OF DIFFERENT TYPES OF NETWORK CABLES

Aim:

To study different types of network cables.

a. understand different types of network cables.

1. unshielded Twisted Pair
2. Shielded Twisted Pair
3. coaxial cable
4. fibre optic cable.

UTP:

category -3 10bps

category-5 upto 100 mbps

category 5e 1Gbps

Applications:

10 Base - T Ethernet

Fast Ethernet, Gigabit Ethernet

Fast Ethernet, Gigabit Ethernet.

STP

category 6, 6a	10Gbps	Gigabit Ethernet, 10G Ethernet

SSTP

category 2	10Gbps	Gigabit Ethernet, 10G Ethernet (100m)

Advantages:

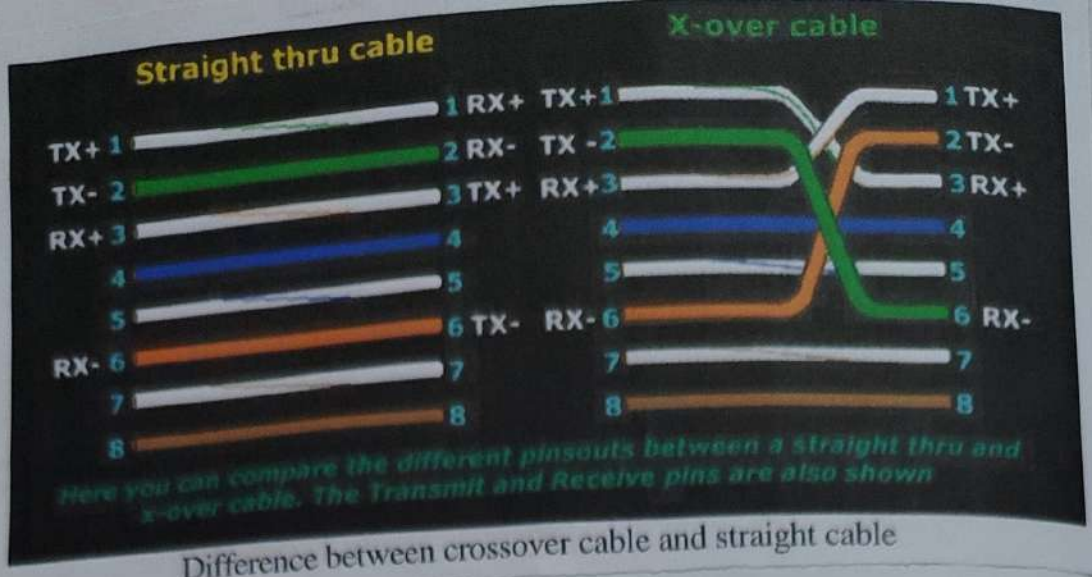
Shielded

Faster than UTP

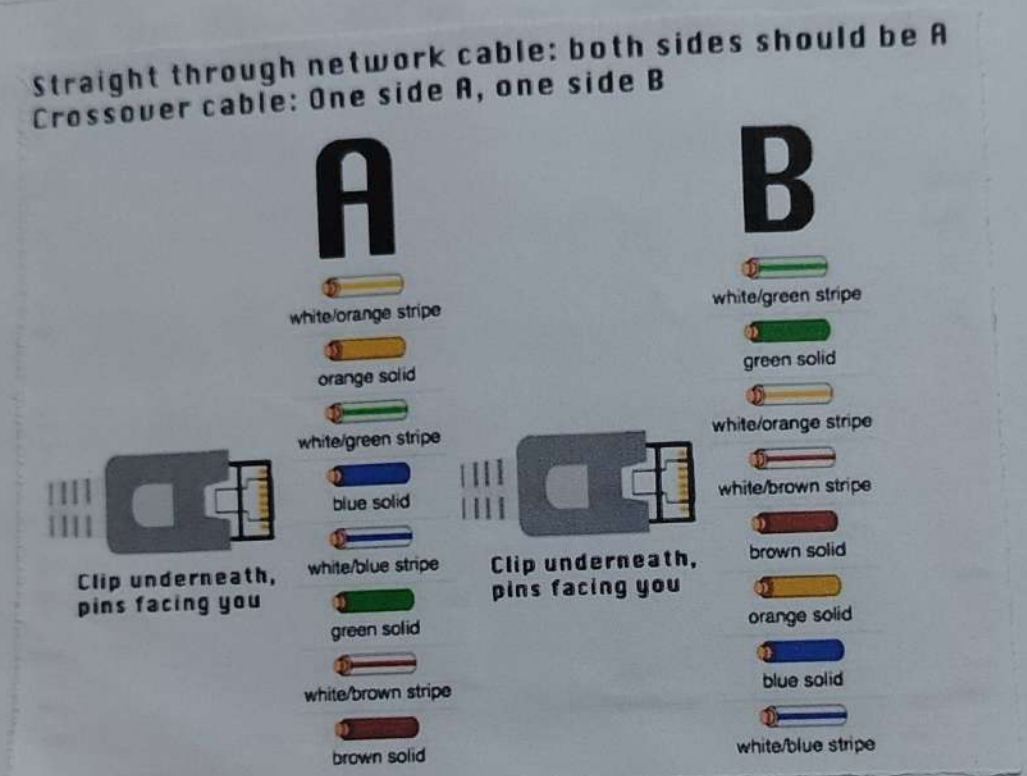
Disadvantages:

Expensive

Greater Installation Effort.



Difference between crossover cable and straight cable





Coaxial Cable:

RG-6 RG-9 RG-11	10-100mbps	Speed of signal in 500m Television network.

Advantages:

High bandwidth
Immune to Interference
Low Loss Bandwidth
Versatile

Disadvantages:

Limited Distance
cost
Size & bulkes

Fibre optics cable:

Single mode Bulk mode	100 Mbps	Maximum distance of fibre optic cable is around 100 meters.

Advantages:

High speed
High Bandwidth
High security
Long distance

Disadvantages:

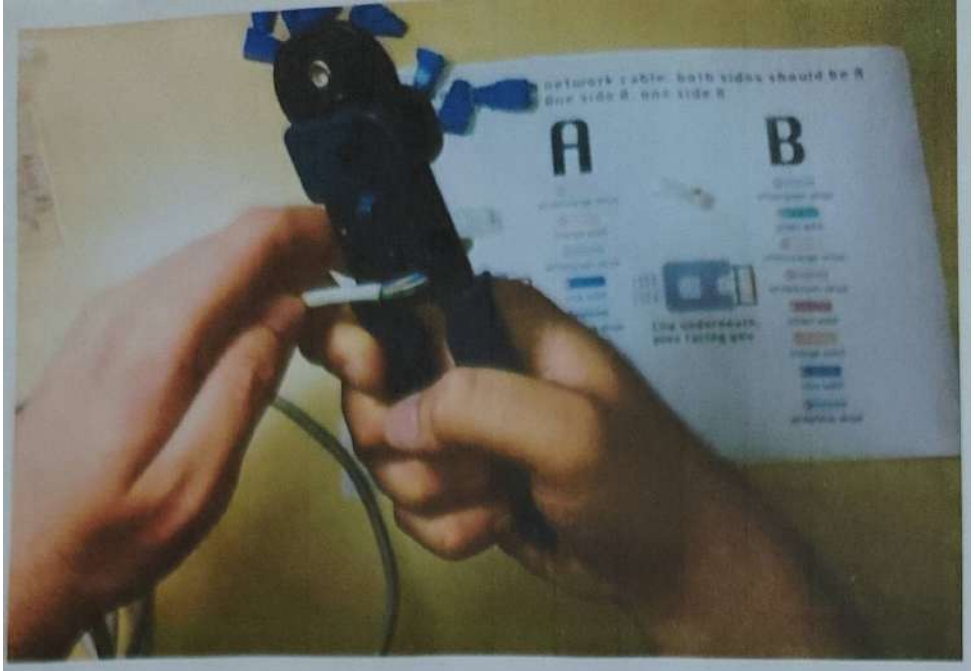
Expensive.

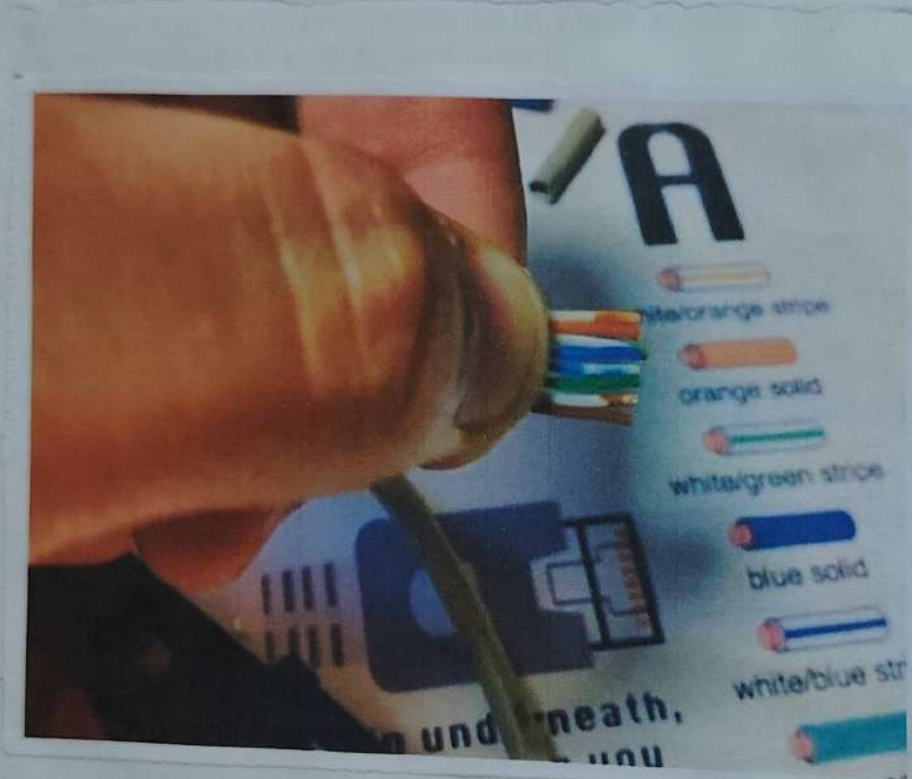
b. Make your own Ethernet:

Step 1: To start construction of the device, begin by threading shields onto the cable.

Step 2: Next step approximately 1.5cm from the cable shielding from both ends.

Step 3: After, you will need to untangle the wire, there should be your "twisted pairs".


A
B


A
orange solid
blue solid


white/green stripe
green solid
white/orange stripe

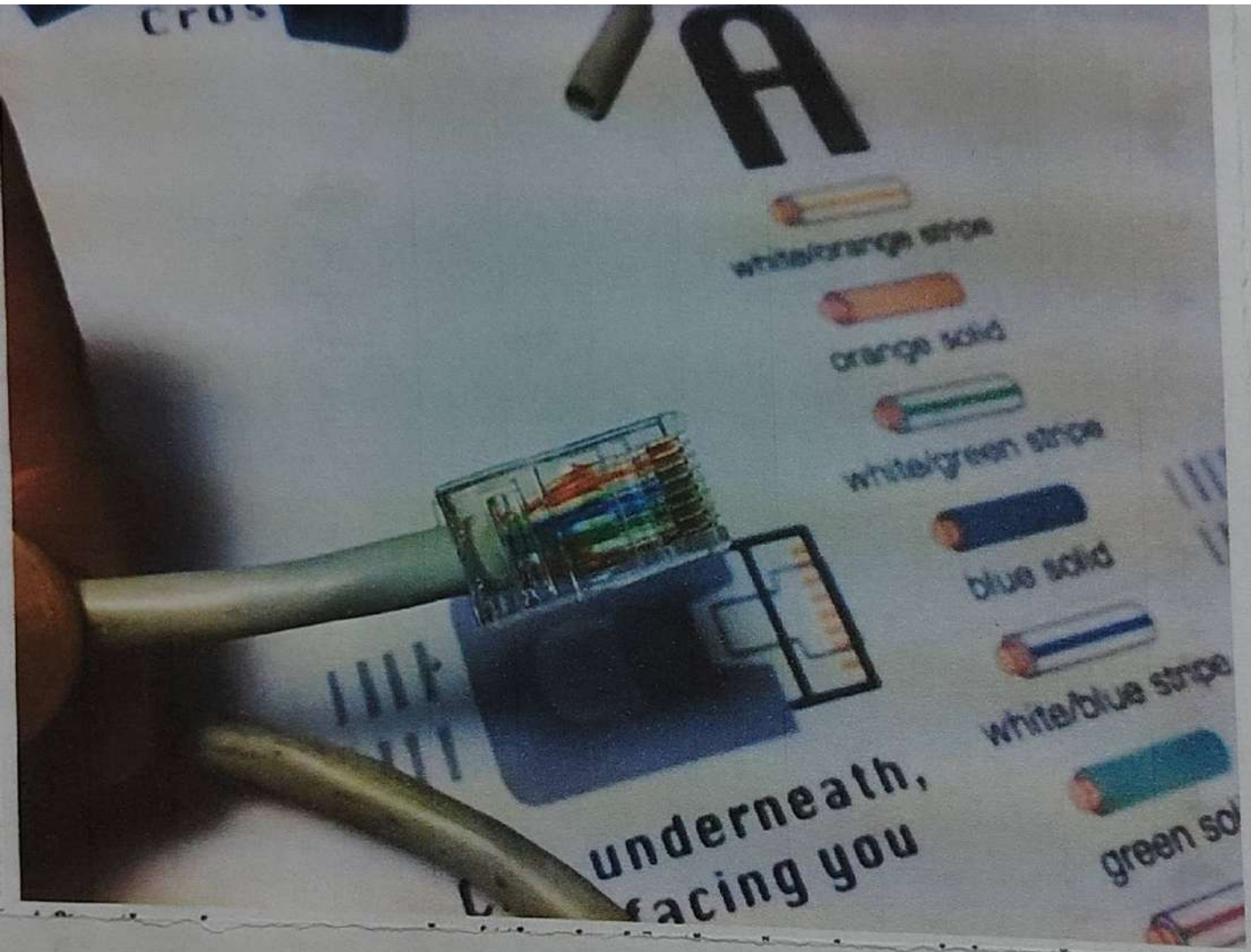
Step 4: Once the order is correct, bunch them together in a bind; and if there are way that stretch and further than to wrote on each end.

Step 5: Next, push the cable sight in, the sheaths and the end of the play needs to be just cover the cable.

Step 6: Lastly, repeat the other end, to make a crossover cable.

Result:

Thus study of different type of wires and working over own ethernet has been studied completely.


A
orange solid
blue solid
underneath,
facing you

EXP: NO: 4
DATE: 6/8/26

STUDY OF LOCAL AREA NETWORK USING A SWITCH AND ETHERNET CABLES

Aim:

To set up and configure a local area network using a switch and ethernet cables and verify communication between connected computers.

Procedure:

Plan the network by arranging 4 computers, one network switch and ethernet cable.

Connect each computer to the switch using ethernet cables.

open control panel → Network → Internet → Network computers.

Right click Ethernet → Properties → TCP/IPV4 → properties

Select use the following IP address and assign.

PC1: 10:1.1.1, Submit mask: 2250.0.0

PC2: 10.1.1.2, Subnet mask: 255.0.0.0

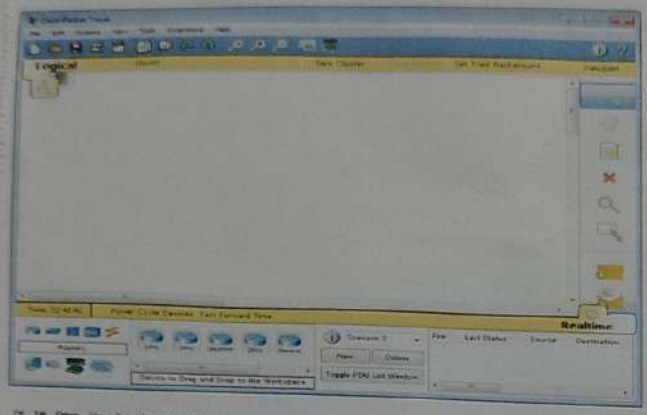
PC3: 10.1.1.3, Subnet mask: 255.0.0.0

PC4: 10.1.1.4, Subnet mask: 255.0.0.0

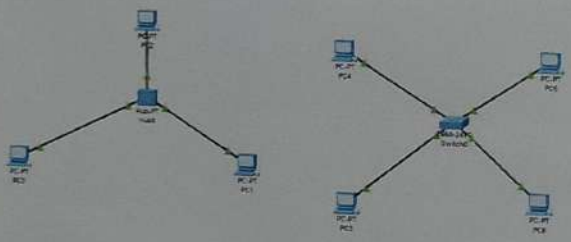
configure the switch by connecting pt to a computer, accessing gt, web interface and assigning the switch IP as 10.1.1.5 with 255.0.0.0.

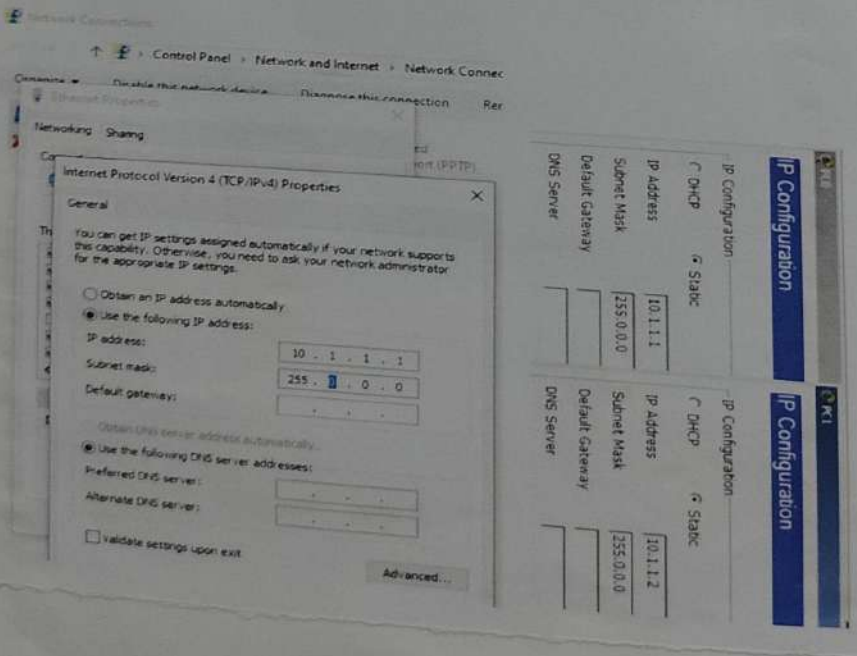
Result:

Thus, a local area network (LAN) was successfully setup and configured using a network switch and ethernet cables.








Control Panel
Network and Internet
Network Connec
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address:
10 . 1 . 1 . 1
Subnet mask:
255 . 0 . 0 . 0
Default gateway:
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.1
255.0.0.0
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.2
255.0.0.0

EXP: NO: 5

DATE: 6/8/26

PACKET CAPTURE USING WIRE SHARK

Aim:

To capture filter inspect and analyse network packets using the wireshark packet Analyzer.

Algorithm:

Open wireshark.

Select the active network interface.

configure capture to stop after 100 packets.

Start packet capture.

Apply display filters.

Inspect packet details and view the flow graph

Save the captured packets.

Procedure:

Launch wireshark and select the active network interface.

Go to capture → options and set stop capture after 10 packets.

click start to assign packet capture

TCP/UDP : tcp || udp

ARP : arp

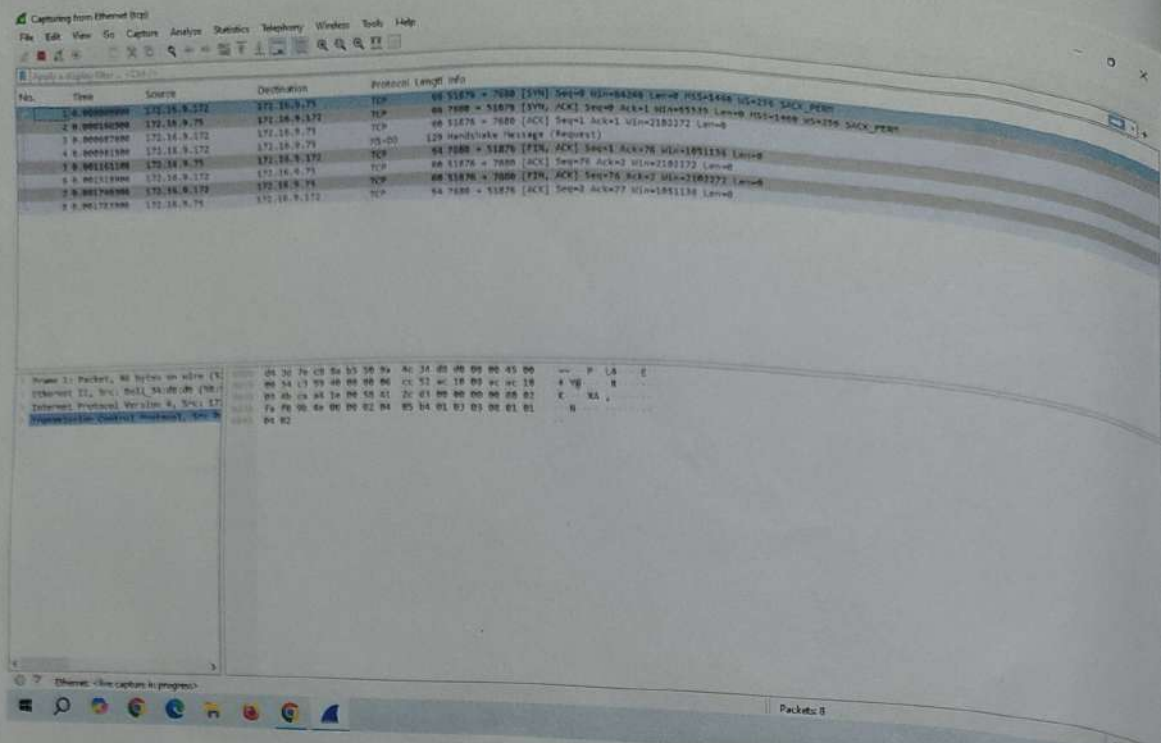
DNS : dns

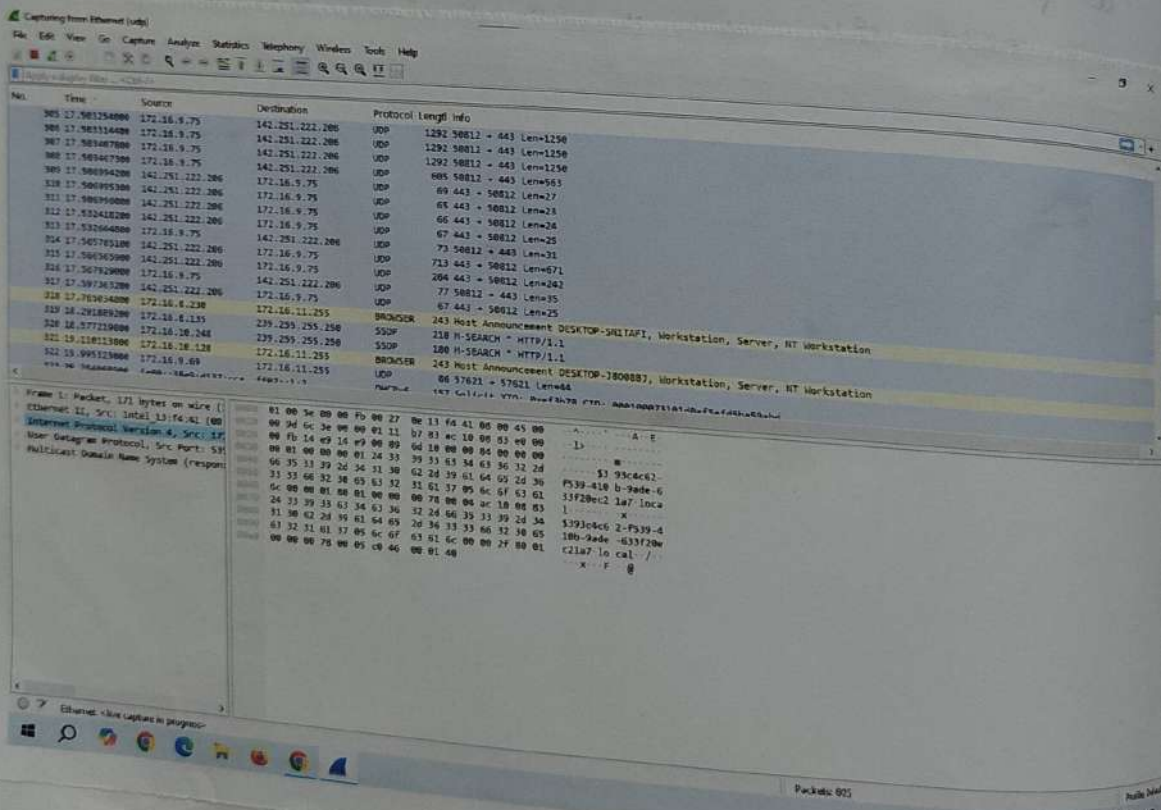
HTTP : http

DHCP : bootp

Inspect the filt packets in the packet list, details and bytes panes.

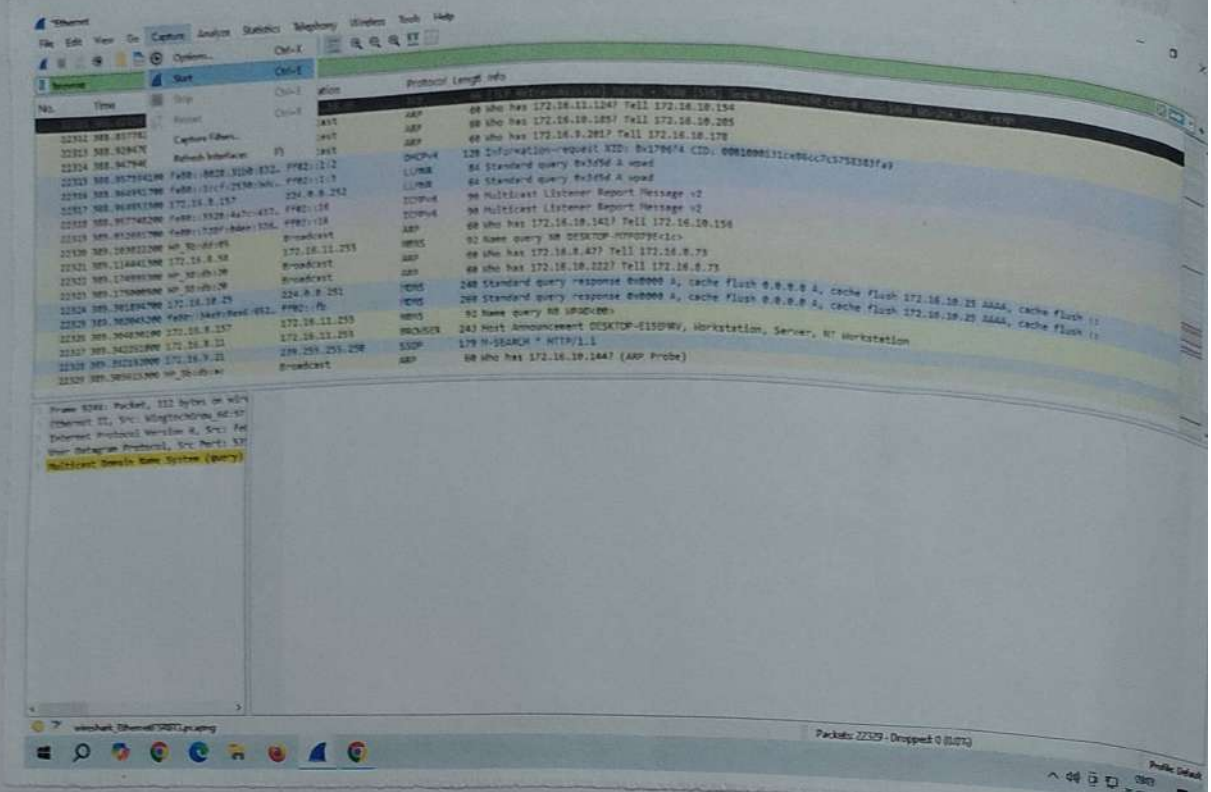
Save the capture file or future analysis.

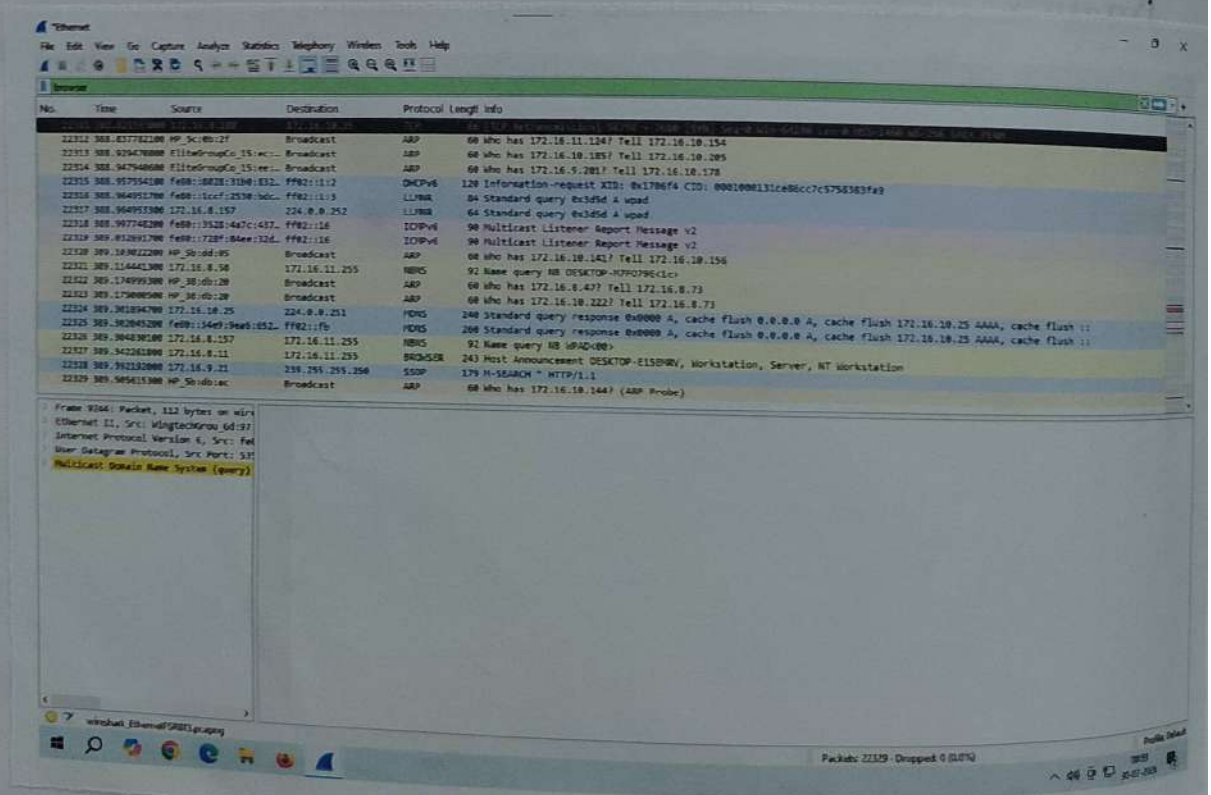




Result:

Successfully captured 100 network packets, applied display filters for TCP, UDP, ARP, DNS, HTTP, IP, ICMP, DHCP analyzed packets details using wireshark.


"Ethernet
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Capture
Options... Ctrl+K
Start Ctrl+E
Stop Ctrl+E
Restart Ctrl+R
Capture Filters...
Refresh Interfaces F5
browse
No. Time Source Destination Protocol Length Info
22312 368.837782100 HP_9c:d0:2f Broadcast ARP 60 Who has 172.16.11.124? Tell 172.16.10.154
22313 368.929478000 EliteGroupCo_15:ec:... Broadcast ARP 60 Who has 172.16.10.185? Tell 172.16.10.205
22314 368.947940600 EliteGroupCo_15:ee:... Broadcast ARP 60 Who has 172.16.9.201? Tell 172.16.10.178
22315 368.957554100 fe80::8028:3160:832... ff02::1:2 DHCPv6 120 Information-request XID: 0x1786f4 CID: 0001000131ce86cc7c5758383fa9
22316 368.964951700 fe80::1ccf:2530:bdc... ff02::1:3 LLMNR 84 Standard query 0x3d5d A wpad
22317 368.964951300 172.16.8.157 224.0.0.252 LLMNR 64 Standard query 0x3d5d A wpad
22318 368.997748200 fe80::3528:4a7c:437... ff02::16 ICMPv6 90 Multicast Listener Report Message v2
22319 369.012691700 fe80::728f:84ee:32d... ff02::16 ICMPv6 90 Multicast Listener Report Message v2
22320 369.169022200 HP_9b:dd:05 Broadcast ARP 60 Who has 172.16.10.141? Tell 172.16.10.156
22321 369.214441300 172.16.8.50 172.16.11.255 NBNS 92 Name query NB DESKTOP-H7FO79E<1c>
22322 369.174999300 HP_38:db:20 Broadcast ARP 60 Who has 172.16.8.47? Tell 172.16.8.73
22323 369.175000500 HP_38:db:20 Broadcast ARP 60 Who has 172.16.10.222? Tell 172.16.8.73
22324 369.301894700 172.16.10.25 224.0.0.251 MDNS 240 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush ::
22325 369.302045200 fe80::154e9:9ea6:852... ff02::fb MDNS 260 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush ::
22326 369.304830100 172.16.8.157 172.16.11.255 NBNS 92 Name query NB WPAD<00>
22327 369.342261000 172.16.8.11 172.16.11.255 BROWSER 243 Host Announcement DESKTOP-E15EHRV, Workstation, Server, NT Workstation
22328 369.392192000 172.16.9.21 239.255.255.250 SSDP 179 M-SEARCH * HTTP/1.1
22329 369.505615300 HP_9b:db:ac Broadcast ARP 60 Who has 172.16.10.144? (ARP Probe)
Frame 9264: Packet, 112 bytes on wire
Ethernet II, Src: WingtechGrou_6d:97
Internet Protocol Version 4, Src: Fe
User Datagram Protocol, Src Port: 53
Multicast DOmain Name System (query)
Packets: 22329 · Dropped: 0 (0.0%)
Profile: Default


"Ethernet
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
browse
No. Time Source Destination Protocol Length Info
22312 368.837782100 HP_9c:d0:2f Broadcast ARP 60 Who has 172.16.11.124? Tell 172.16.10.154
22313 368.929478000 EliteGroupCo_15:ec:... Broadcast ARP 60 Who has 172.16.10.185? Tell 172.16.10.205
22314 368.947940600 EliteGroupCo_15:ee:... Broadcast ARP 60 Who has 172.16.9.201? Tell 172.16.10.178
22315 368.957554100 fe80::8028:3160:832... ff02::1:2 DHCPv6 120 Information-request XID: 0x1786f4 CID: 0001000131ce86cc7c5758383fa9
22316 368.964951700 fe80::1ccf:2530:bdc... ff02::1:3 LLMNR 84 Standard query 0x3d5d A wpad
22317 368.964951300 172.16.8.157 224.0.0.252 LLMNR 64 Standard query 0x3d5d A wpad
22318 368.997748200 fe80::3528:4a7c:437... ff02::16 ICMPv6 90 Multicast Listener Report Message v2
22319 369.012691700 fe80::728f:84ee:32d... ff02::16 ICMPv6 90 Multicast Listener Report Message v2
22320 369.169022200 HP_9b:dd:05 Broadcast ARP 60 Who has 172.16.10.141? Tell 172.16.10.156
22321 369.214441300 172.16.8.50 172.16.11.255 NBNS 92 Name query NB DESKTOP-H7FO79E<1c>
22322 369.174999300 HP_38:db:20 Broadcast ARP 60 Who has 172.16.8.47? Tell 172.16.8.73
22323 369.175000500 HP_38:db:20 Broadcast ARP 60 Who has 172.16.10.222? Tell 172.16.8.73
22324 369.301894700 172.16.10.25 224.0.0.251 MDNS 240 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush ::
22325 369.302045200 fe80::154e9:9ea6:852... ff02::fb MDNS 260 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush ::
22326 369.304830100 172.16.8.157 172.16.11.255 NBNS 92 Name query NB WPAD<00>
22327 369.342261000 172.16.8.11 172.16.11.255 BROWSER 243 Host Announcement DESKTOP-E15EHRV, Workstation, Server, NT Workstation
22328 369.392192000 172.16.9.21 239.255.255.250 SSDP 179 M-SEARCH * HTTP/1.1
22329 369.505615300 HP_9b:db:ac Broadcast ARP 60 Who has 172.16.10.144? (ARP Probe)
Frame 9264: Packet, 112 bytes on wire
Ethernet II, Src: WingtechGrou_6d:97
Internet Protocol Version 4, Src: Fe
User Datagram Protocol, Src Port: 53
Multicast DOmain Name System (query)
Packets: 22329 · Dropped: 0 (0.0%)
Profile: Default

EXP : 6
DATE : 8/8/26

Error Detection and correction using Hamming code.

Aim :

To write a program to implement error detection and correction using the hamming code.

Algorithm :

1. Start the program.
2. Read the number of 4-digit data blocks.
3. For each digit 4 bit data block (n) perform hamming.

$P_1 = d_1 XOR d_2 XOR d_4$

$P_2 = d_1 XOR d_3 XOR d_4$

$P_3 = d_2 XOR d_3 XOR d_4$

4. Simulate transmission by introducing a single bit error.
5. $C_1 = P_1 XOR d_1 XOR d_4$; $C_2 = P_2 XOR d_1 XOR d_3 XOR d_4$;

$C_4 = P_1 XOR d_2 XOR d_3 XOR d_4$

6. Compute error position $= C_1 C_2 C_4$
7. If error position $= 0$, no error has occured.
8. Extract the original data bits from the corrected bit
9. Repeat step 3 to 8 for all n-blocks in the data
10. Stop the program.

Program :

```
def encoded(d):
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[0] ^ h[6]
```

```
    h[3]=h[4]^h[5]^h[6]
    return h

def detect_and_correct(h):
    c1 = h[0]^h[2]^h[4]^h[6]
    c2 = h[1]^h[2]^h[5]^h[6]
    c4 = h[3]^h[4]^h[5]^h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        h[error_pos-1] ^= 1
    else:
        print("No error")
    return error_pos

def main():
    n = int(input("Enter number: "))
    for b in range(1,n+1):
        d = list(map(int, input("Enter 4 data bits").split
        h = encode(d)
        print(f"original data bits : {to_str(d)}")
        print(f"Encoded Hamming code : {to_str(h)}")
        if 1<=pos<=7:
            h[pos-1] ^= 1
            print(f"Transmitted Code (with error
                                induced):
                {to_str(h)}")
        else
            print(f"Transmitted Code : {to_str(h)}")
        print(f"received code : {to_str(h)}")
        detect_and_correct(h)
        print(f"corrected code : {to_str(h)}")
        rd = [h[2], h[4], h[5], h[6]]
        print(f"recovered data bits : {to_str(rd)}")
```

```
if __name__ == "__main__":
    main()
```

Result:

The program to implement error detection and correction using the Hamming code concept was successfully executed and verified.

output:

Enter number of 4 bit data blocks: 2

--Block1--

Enter 4 data bits: 1011

Original hamming code: 1011

Encoded hamming code: 0110011

Enter bit position to introduce error: 3

Transmitted code: 0100011

Received code: 0100011

corrected code: 0110011

recovered code data bits: 1011

--Block2--

Enter 4 data bits: 0110

Original data bits: 0110

Encoded hamming code: 1100110

Enter bit position to introduce error: 6

Transmitted code: 1100100

Received code: 1100100

Error detected at bit position: 6

Corrected code: 1100110

recovered data bits: 0110

EXP. NO : 7
DATE : 13/8/26

Flow control of data link layer using Sliding window Protocol

Aim :

To write a program to implement flow control at the data link layer using sliding window protocol.

Algorithm :

1. Start the program
2. Read the total no of frames to be sent (N), the sender is window size (ws).
3. Initialize base = 0
4. Repeat while base < N
   a. Determine the current sender window as the range (base, base+W-1)
   b. Send all the frames with sequence number from next seq.
   c. If the last frame lies within the frames just sent.
      i) The receiver discards the last frame and any subsequent.
      ii) Then sender times out and retransmits all frames.
   d. Else
      i) The receiver accepts each frame in order and sends a cumulative acknowledgment.
      ii) On receiving the ACK for the last frame sent, the sender slides the window forward.
5. Repeat steps until all N frames transmitted.
6. Stop.

output:

Enter total no of frames to send : 8

Enter sender's window size : 4

Enter frame no to simulate as LOST : 5

Sender window : [frame 0 - frame 3]

Node A → sending frame 0

Node A → sending frame 1

Node A → sending frame 2

Node A → sending frame 3

Node B → Received frame 0, sending ACK 0

Node B → received frame 1, sending ACK 1

Node B → received frame 2, Sending ACK 2

Node B → received frame 3, sending ACK 3

Sender window : [frame 4 - frame 7]

Node A → sending frame 4

Node A → sending frame 5

Node A → sending frame 6

Node B → frame 5 missing

Node A → Timeout : resending window starting from frame 5

```
def simulate(total_frames, window_size, lost_frame):
    base = 0
    next_seq = 0
    while base < total_frames:
        window_end = min(base + window_size - 1, total_frames - 1)
        print(f"Sender window: [{base} - {window_end}]")

        while next_seq < base + window_size and next_seq < total_frames:
            if next_seq == lost_frame:
                print(f"Frame {next_seq} lost in transit!")
            next_seq += 1

        if lost_frame is not None and base <= lost_frame < next_seq:
            print(f"Node B --> Frame {lost_frame} missing. Discarding out of order frames after it.")
            next_seq = lost_frame
            lost_frame = None

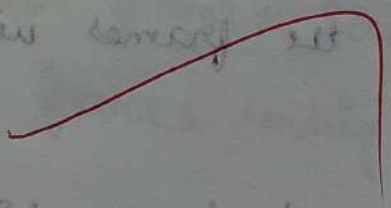
        for f in range(base, next_seq):
            print(f"Node A --> ACK {next_seq - 1} received. Sliding window forward.")

def main():
    total_frames = int(input("Enter total no of frames to send: "))
    lost_frame = int(input())
    if lost_frame == -1:
        lost_frame = None
    simulate(total_frames, window_size, lost_frame)
```

Sender window: [frame 4 – frame 7]

Node A → Sending frame 5
Node A → Sending frame 6
Node A → Sending frame 7
Node B → Received frame 4, sending ACK 4
Node B → received frame 5, sending ACK 5
Node B → received frame 6, sending ACK 6
Node A → ACK 7 received sliding window forward

All 8 frames successfully transmitted from node A to node B.



```
if __name__ == "__main__":
    main()
```

Result:

The program to implement flow control at the data link layer using the sliding window protocol was successfully executed and verified by simulating the flow of frames between two nodes.

EXP: NO : 9
Date : 24/8/26

Implementation of Subnetting in cisco packet tracer.

Aim:

To implement classless IP Subnetting using the cisco packet tracer simulator and test connectivity between different subnets.

Requirements:

- Cisco packet Tracer
- Routers
- Switches
- PCs
- Ethernet cables.

Procedure:

1. Creating the Network Topology:
   - open cisco packet tracer
   - create a new workspace
   - Add the required routers, switches and PCs.
   - connect the devices using appropriate Ethernet cable.
2. IP Addressing:

LAN1	R1G0/0	192.168.1.1/27
LAN2	R1G0/1	192.168.2.1/27
LAN3	R2F0/0	192.168.3.1/27
LAN4	R2F0/1	192.168.4.1/27

For example;

IP Address: 192.168.1.11

Subnet mask: 255.255.255.224

Default gateway: 192.168.1.1

3. configuring Router R1

open R1 → CLI and enter:

enable

configure terminal

interface gigabit Ethernet 0/10

IP address 192.168.1.1.255.255.255.244

no shutdown.

exit

4. configuring Router R2

enable

configure terminal

interface fast ethernet 0/10

IP address 192.168.3.255.255.224

no shutdown

exist.

5. configuring the ethernet.

For S1:

enable

configure terminal

interface fast Ethernet 0/1

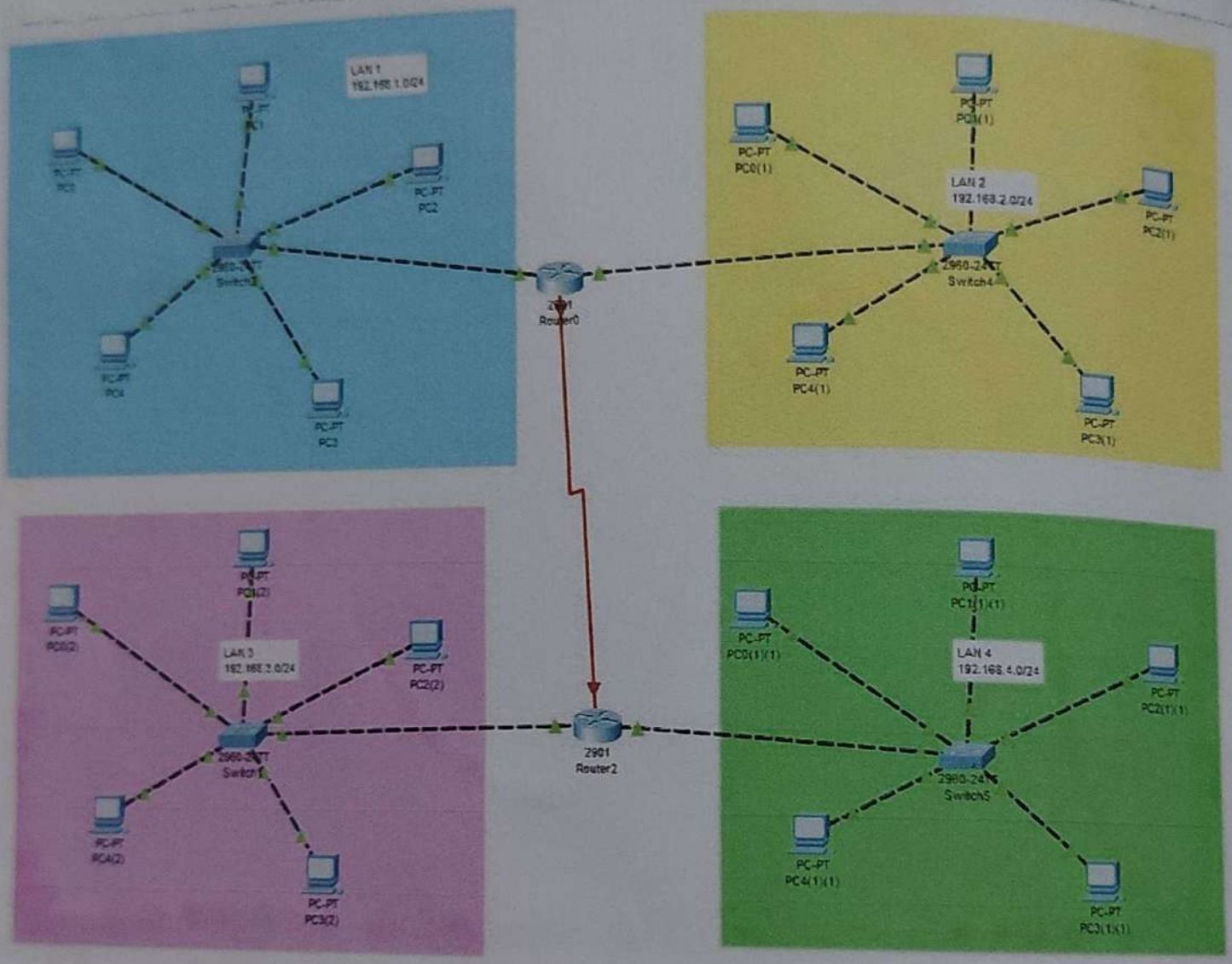
Switch port mode access.

Exit.

6. configuring the PCs:

open each PC → Desktop → IP configuration and enter the appropriate IP address subnet mask and default gateway.

PC	IP address	Subnet mask	Default gateway
PC1	192.168.1.11	255.255.255.244	192.168.1.1
PC2	192.168.1.12	255.255.255.244	192.168.1.1


LAN 1
192.168.1.0/24
PC-PT
PC1
PC-PT
PC0
PC-PT
PC2
PC-PT
PC4
PC-PT
PC3
2901
Router0
LAN 2
192.168.2.0/24
PC-PT
PC0(1)
PC-PT
PC1(1)
PC-PT
PC2(1)
PC-PT
PC4(1)
PC-PT
PC3(1)
LAN 3
192.168.3.0/24
PC-PT
PC0(2)
PC-PT
PC1(2)
PC-PT
PC2(2)
PC-PT
PC4(2)
PC-PT
PC3(2)
2901
Router2
LAN 4
192.168.4.0/24
PC-PT
PC0(1)(1)
PC-PT
PC1(1)(1)
PC-PT
PC2(1)(1)
PC-PT
PC4(1)(1)
PC-PT
PC3(1)(1)

Testing the Network:

1. Open PC → Desktop → command prompt

   Ping 192.168.1.1

2. Test connectivity blw PCs in the subnet: 192.168.1.12

Result:

Thus the Implementation of subnetting in cisco Packet tracer was executed & verified successfully.

EXP: NO: 10
Date: 24/8/26

a) Interconnecting using Routers.

Aim:

To design and configure a simple internetwork using a router and two in CISCO packet Tracer and test connectivity using PPU.

Step 1: configure Router 1

1) open cisco packet tracer
2) select Router → Router 1
3) open CLI and press enter
4) Enter
   enable
   Config t
5) configure Fast Ethernet 0/0:
   Interface Fast Ethernet 0/0
   IP address 192.168.10.1255.255.255.0
   NO shut down.
6) Configure Fast Ethernet 0/0:
   Interface Fas Ethernet 0/1
   IP address 192.168.10.255.255.255.0.
   NO shutdown.

Step 2: Configure PC0

Go to PC0 → desktop → IP configuration
IP address: 192.168.10.2
Subnet Mask: 255.255.255.0
Default Gateway: 192.168.10.1

Step 3: configure PC1

Go to PC1 → desktop → IP configuration

IP Address: 192.168.20.2

Subnet mask: 255.255.255.0

Default Gateway: 198.168.20.1

Step 4: connect the devices

use copper straight-through cable

PC0 Fast Ethernet 0 → Router1 Fast Ethernet 0/0.

PC1 Fast Ethernet 0 → Router1 Fast Ethernet 0/1.

Step 5: Test connectivity:

1) Select simulation mode.

2) choose Add simple PDU.

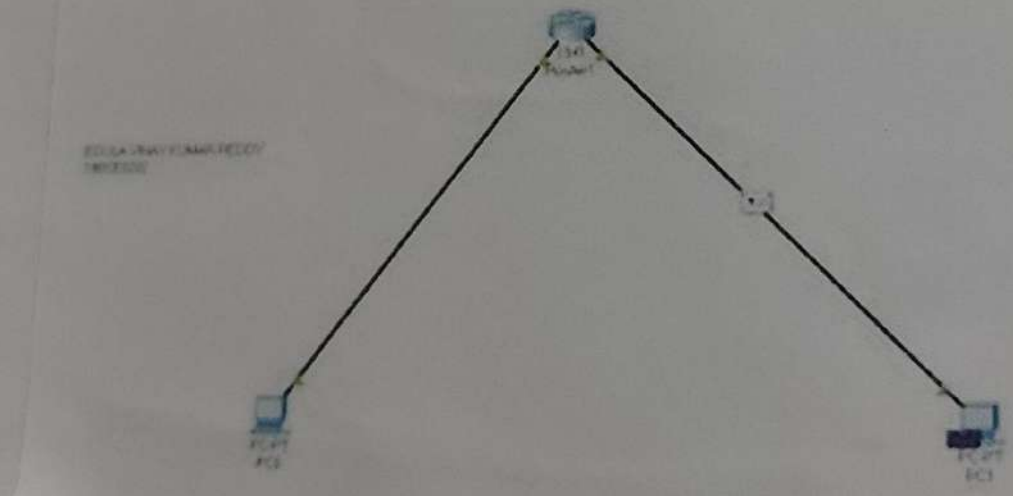
3) click PC0 → PC1

4) If the PDU is successful, the two network are communicating through the router.

Result:

The internetwork was successfully designed and configured, and connectivity between PC0 and PC1 was verified.





EXP. No: 10 (b)
Date: 3d 8/26

Wireless Router, DHCP Server and Internet cloud.

Aim:

To design and configure and internetwork using a wireless router, DHCP Servers and Internet cloud and verify connectivity.

Step 1: Build the Network.

Add:

PC
wireless Router
laptop
cable Modem
Internet cloud
cisco.com Server

Step 2: Connect Devices

use the appropriate cables

PC → wireless router → cable modem:

copper straight - through

wireless Router → cable modem:

copper straight - through.

cable modem → Internet cloud

Coaxial cable.

Internet cloud → cisco.com Server

copper straight - through

Step 3: configure wireless router

1) open wireless Router → GUI
2) Go to wireless.
3) Set DNS Server as 208.67.220.200.22
4) click save settings.

Step 4:

Configure Cisco.com Server.

DHCP

DHCP : ON

PoolName : DHCP Pool

default Gateway : 208.51.220.220

Starting IP : 208.51.220.1

Maximum users : 50

click add

Step 5 : Test connectivity

1) Open Pc → Desktop → command prompt

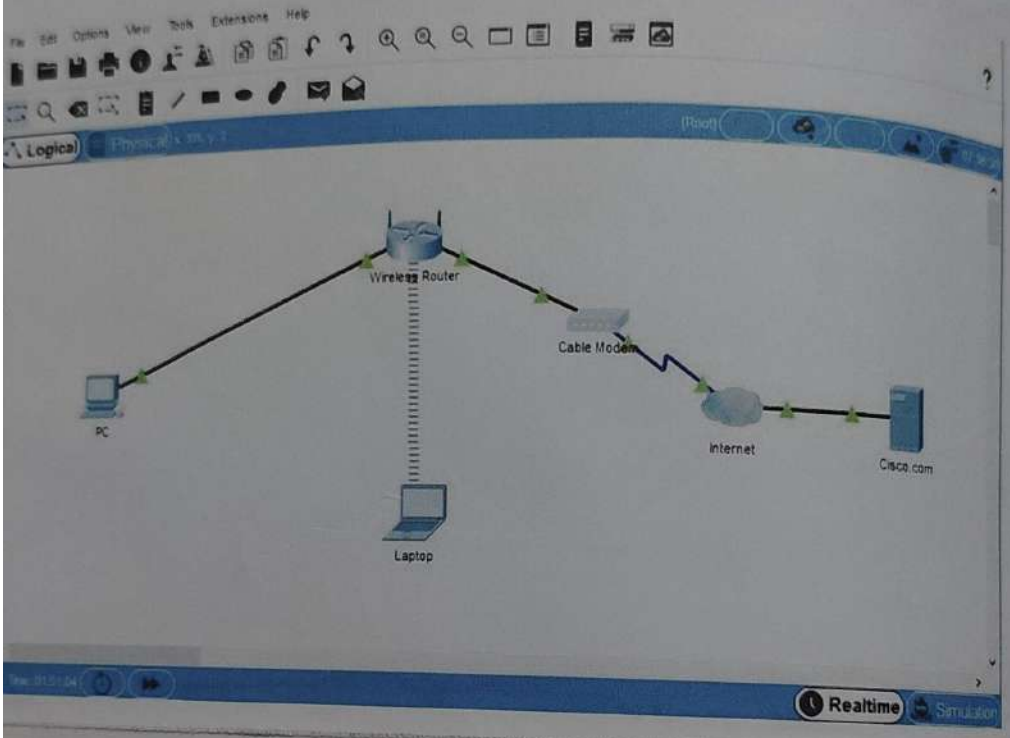
2) Type : ipconfig /release.

3) Then : ipconfig /renew

4) Test the server : ping cisco.com

Result :

The wireless internet work was successfully designed using a wireless router DHCP server and internet cloud, and connectivity was verified using the Ping command.


File Edit Options View Tools Extensions Help
Logical
Wireless Router
Cable Modem
PC
Internet
Cisco.com
Laptop
Realtime
Simulation

Exp. No: 11
Date: 3d/8/26

Routing at Network layer.

a) Simulate static routing protocol configuration using Cisco Packet Tracer.

AIM:

To simulate and configure static routing between multiple networks using cisco packet tracer, configure primary and backup static routers using administrative distance.

Requirements:

Cisco Packet Tracer.

Three routers - Router 0, Router 1,2

PCs/end devices.

Properly assigned IP address.

Observation:

A network topology consisting of three routers, routers 1,2 was created in Cisco Packet Tracer, different networks were connected and were assigned suitable IP address.

Router 1:

Router 1 was configured with static routers to the 10.0.0.0/8 to 40.0.0.0/8 networks. The route through router 0 was assigned a lower administrative distance primary route. The configuration was verified using show IP route static.

Router 2:

was configured with static routes to the

table and verified using show ip router static.

Backup route verification:

The main link b/w router R1 was disconnected to simulate a link failure connectivity was verified using ping & tracer.

verification:

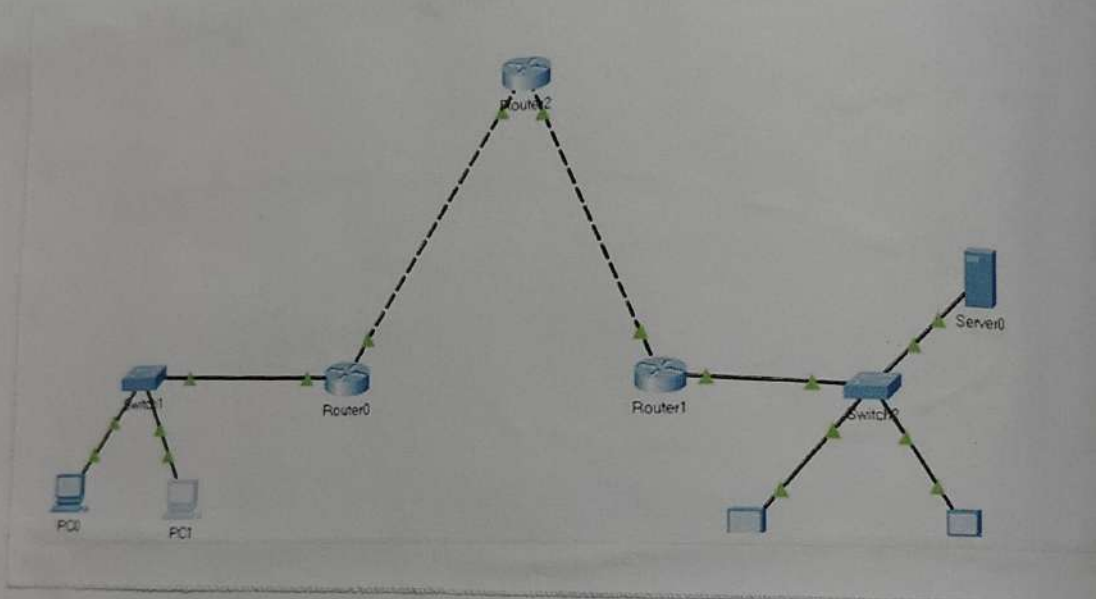
show ip router static The routing tables showed the configured static routes & their administrative using the ping command.

Deleting a static route:

Show ip route static. After deleting the main route, the backup route becomes the active.

Result:

Thus, static routing was successfully configured & simulated using cisco packet tracer.


Router2
Server0
Router0
Router1
PC0
PC1

Exp: No: 12
Date: 22/9/26

Implement echo client server using TCP/UDP sockets.

Aim:

To write and implement an echo client server program using TCP sockets inc.

Result:

Thus, the echo client server was successfully implemented using sockets.

Exp 12

Part A: Echo Client-Server using TCP Sockets

AIM

To write and implement an Echo Client-Server program using TCP sockets in C.

Algorithm

Server Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Put the server socket in listening mode using listen().
4. Accept an incoming client connection using accept().
5. Read the message sent by the client using read() or recv().
6. Echo the same message back to the client using write() or send().
7. Close the client and server sockets using close().

Client Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Connect to the server's IP address and port number using connect().
3. Prompt the user for input and send the message to the server using write() or send().
4. Receive the echoed message back from the server using read() or recv().
5. Display the received message on the terminal.
6. Close the socket using close().

Code Implementation (TCP)

`tcp_server.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 1. Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
```

```
    exit(EXIT_FAILURE);
  }

  // 2. Bind socket
  address.sin_family = AF_INET;
  address.sin_addr.s_addr = INADDR_ANY;
  address.sin_port = htons(PORT);

  if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
    perror("Bind failed");
    exit(EXIT_FAILURE);
  }

  // 3. Listen for connections
  if (listen(server_fd, 3) < 0) {
    perror("Listen failed");
    exit(EXIT_FAILURE);
  }

  printf("[TCP Server] Waiting for incoming connections on port
%d...\n", PORT);

  // 4. Accept connection
  if ((new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen)) < 0) {
    perror("Accept failed");
    exit(EXIT_FAILURE);
  }

  // 5 & 6. Read and Echo back
  read(new_socket, buffer, BUFFER_SIZE);
  printf("[TCP Server] Received from client: %s\n", buffer);

  send(new_socket, buffer, strlen(buffer), 0);
  printf("[TCP Server] Echoed message back to client.\n");

  // 7. Close sockets
  close(new_socket);
  close(server_fd);
  return 0;
}
```

tcp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
```

```
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];

    // 1. Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <=
0) {
        printf("\nInvalid address / Address not supported \n");
        return -1;
    }

    // 2. Connect to server
    if (connect(sock, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // 3. Send message
    printf("Enter message to send to server: ");
    fgets(message, BUFFER_SIZE, stdin);
    message[strcspn(message, "\n")] = 0; // Remove newline

    send(sock, message, strlen(message), 0);
    printf("[TCP Client] Message sent.\n");

    // 4 & 5. Receive echoed message and display
    read(sock, buffer, BUFFER_SIZE);
    printf("[TCP Client] Echo received from server: %s\n", buffer);

    // 6. Close socket
    close(sock);
    return 0;
```

}

Input/Output (TCP)

Input (Client Terminal):

Enter message to send to server: Hello Networks Lab!

Output (Server Terminal):

[TCP Server] Waiting for incoming connections on port 8080...

[TCP Server] Received from client: Hello Networks Lab!

[TCP Server] Echoed message back to client.

Output (Client Terminal):

[TCP Client] Message sent.

[TCP Client] Echo received from server: Hello Networks Lab!

Part B: Echo Client-Server using UDP Sockets

AIM

To write and implement an Echo Client-Server program using UDP sockets in C.

Algorithm

Server Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Receive a message and store the client's address using recvfrom().
4. Echo the message back to the client using sendto().
5. Close the socket using close().

Client Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Get input from the user and send the message to the server using sendto().
3. Receive the echoed message back from the server using recvfrom().
4. Display the received message on the terminal.
5. Close the socket using close().

Code Implementation (UDP)

udp_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE] = {0};
    struct sockaddr_in servaddr, cliaddr;
```

socklen_t len

```
socklen_t len = sizeof(cliaddr);

// 1. Create socket
if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}

memset(&servaddr, 0, sizeof(servaddr));
memset(&cliaddr, 0, sizeof(cliaddr));

// 2. Bind socket
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = INADDR_ANY;
servaddr.sin_port = htons(PORT);

if (bind(sockfd, (const struct sockaddr *)&servaddr,
sizeof(servaddr)) < 0) {
    perror("Bind failed");
    exit(EXIT_FAILURE);
}

printf("[UDP Server] Waiting for data on port %d...\n", PORT);

// 3. Receive message
int n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE,
MSG_WAITALL, (struct sockaddr *)&cliaddr, &len);
buffer[n] = '\0';
printf("[UDP Server] Received from client: %s\n", buffer);

// 4. Echo back message
sendto(sockfd, (const char *)buffer, strlen(buffer),
MSG_CONFIRM, (const struct sockaddr *)&cliaddr, len);
printf("[UDP Server] Echoed message back to client.\n");

// 5. Close socket
close(sockfd);
return 0;
}
```

udp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024
```

```
int main() {
    int sockfd;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];
    struct sockaddr_in servaddr;
    socklen_t len = sizeof(servaddr);

    // 1. Create socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = INADDR_ANY;

    // 2. Get input and send
    printf("Enter message to send to UDP server: ");
    fgets(message, BUFFER_SIZE, stdin);
    message[strcspn(message, "\n")] = 0;
    sendto(sockfd, (const char *)message, strlen(message),
MSG_CONFIRM, (const struct sockaddr *)&servaddr,
sizeof(servaddr));
    printf("[UDP Client] Message sent.\n");

    // 3 & 4. Receive echo and display
    int n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE,
MSG_WAITALL, (struct sockaddr *)&servaddr, &len);
    buffer[n] = '\0';
    printf("[UDP Client] Echo received from server: %s\n",
buffer);

    // 5. Close socket
    close(sockfd);
    return 0;
}
```

Input/Output (UDP)

Input (Client Terminal):

Enter message to send to UDP server: Testing UDP Socket

Output (Server Terminal):

[UDP Server] Waiting for data on port 8080...

[UDP Server] Received from client: Testing UDP Socket

[UDP Server] Echoed message back to client.

Output (Client Terminal):

[UDP Client] Message sent.

[UDP Client] Echo received from server: Testing UDP Socket

EXP NO: 13
Date: 26/9/26

PING Program.

Aim:

To implement a Ping program using Python raw sockets to check the reachability of a host and calculate the round trip time.

Result:

Thus Ping program using python to check the reachability of a host is calculated successfully

AIM

To implement a custom Ping program using raw ICMP (Internet Control Message Protocol) sockets in C.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Initialize a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_ICMP.
2. **Resolve Host IP:** Convert the domain name or IPv4 string into a valid target IP address using DNS lookup.
3. **Construct ICMP Packet:**
   - Set ICMP type to ICMP_ECHO (Type 8, Code 0).
   - Set Sequence Number and Process Identifier (PID) fields.
   - Calculate and set the Internet Checksum over the entire ICMP header and payload.
4. **Send Echo Request:** Transmit the packet to the destination address using sendto(). Record the transmission timestamp.
5. **Receive Echo Reply:**
   - Wait for incoming packets with recvfrom() within a timeout window.
   - Parse the outer IP header to extract the inner ICMP packet.
   - Verify that the received packet is an ICMP_ECHOREPLY (Type 0) matching the process ID and sequence number.
6. **Calculate Metrics:** Subtract transmission timestamp from receive timestamp to obtain Round Trip Time (RTT).
7. **Display Output:** Print sequence number, TTL (Time to Live), packet size, and RTT. Repeat for desired number of pings.

Code Implementation (ping.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <sys/time.h>

#define PACKET_SIZE 64
#define PORT_NO 0
#define TIMEOUT 1 // Timeout in seconds
```

```
// Standard Internet Checksum Algorithm
unsigned short calculate_checksum(unsigned short *buffer, int
length) {
    unsigned long sum = 0;
    while (length > 1) {
        sum += *buffer++;
        length -= 2;
    }
    if (length == 1) {
        sum += *(unsigned char *)buffer;
    }
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: sudo %s <hostname/IP>\n", argv[0]);
        return 1;
    }

    int sockfd;
    struct sockaddr_in dest_addr;
    struct hostent *host;
    char packet[PACKET_SIZE];
    struct icmp *icmp_hdr;
    struct timeval timeout, tv_start, tv_end;

    // 1. Resolve Hostname
    host = gethostbyname(argv[1]);
    if (host == NULL) {
        perror("Could not resolve hostname");
        return 1;
    }

    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_addr = *((struct in_addr *)host-
>h_addr_list[0]);

    // 2. Create Raw Socket for ICMP
    sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP
    if (sockfd < 0) {
        perror("Socket creation failed (run as root)");
        return 1;
    }
```

```
// Set Receive Timeout
timeout.tv_sec = TIMEOUT;
timeout.tv_usec = 0;
setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, (const
char*)&timeout, sizeof(timeout));

printf("PING %s (%s) %d bytes of data.\n", argv[1],
inet_ntoa(dest_addr.sin_addr), PACKET_SIZE);

for (int seq = 1; seq <= 4; seq++) {
memset(packet, 0, PACKET_SIZE);

// 3. Construct ICMP Header
icmp_hdr = (struct icmp *)packet;
icmp_hdr->icmp_type = ICMP_ECHO;
icmp_hdr->icmp_code = 0;
icmp_hdr->icmp_id = getpid();
icmp_hdr->icmp_seq = seq;
icmp_hdr->icmp_cksum = 0;

// Fill remaining bytes with sample data payload
for (int i = sizeof(struct icmp); i < PACKET_SIZE; i++) {
packet[i] = i + '0';
}

icmp_hdr->icmp_cksum = calculate_checksum((unsigned
short *)packet, PACKET_SIZE);

// 4. Send Packet & Record Time
gettimeofday(&tv_start, NULL);
if (sendto(sockfd, packet, PACKET_SIZE, 0, (struct
sockaddr *)&dest_addr, sizeof(dest_addr)) <= 0) {
perror("Failed to send packet");
continue;
}

// 5. Receive Reply
char recv_buf[1024];
struct sockaddr_in from_addr;
socklen_t from_len = sizeof(from_addr);

int bytes_rcvd = recvfrom(sockfd, recv_buf,
sizeof(recv_buf), 0, (struct sockaddr *)&from_addr, &from_len);
gettimeofday(&tv_end, NULL);

if (bytes_rcvd < 0) {
printf("Request timed out for icmp_seq %d\n", seq);
} else {
```

```
        // Extract IP header to inspect TTL and payload
        struct ip *ip_hdr = (struct ip *)recv_buf;
        int ip_hdr_len = ip_hdr->ip_hl * 4;
        struct icmp *reply_icmp = (struct icmp *)(recv_buf +
ip_hdr_len);

        // Calculate RTT in milliseconds
        double rtt = (tv_end.tv_sec - tv_start.tv_sec) * 1000.0 +
                (tv_end.tv_usec - tv_start.tv_usec) / 1000.0;

        if (reply_icmp->icmp_type == ICMP_ECHOREPLY) {
            printf("%d bytes from %s: icmp_seq=%d ttl=%d
time=%.2f ms\n",
                bytes_rcvd - ip_hdr_len,
                inet_ntoa(from_addr.sin_addr),
                reply_icmp->icmp_seq,
                ip_hdr->ip_ttl,
                rtt);
        }
    }
    sleep(1); // Wait 1 second between requests
  }

  close(sockfd);
  return 0;
}
```

Input / Output

Input

Command Terminal:

sudo ./myping 127.0.0.1

Output

Server Terminal:

PING 127.0.0.1 (127.0.0.1) 64 bytes of data.

64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.08 ms

64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.06 ms

64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.07 ms

64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.05 ms

EXP: NO: 14
Date: 3/9/26

Network Packet Sniffer using Raw Sockets.

Aim:

To write and implement a network packet sniffer using raw sockets in C to capture and parse -- IP, TCP and UDP.

Result:

Thus the network sniffer using raw sockets is implemented successfully to capture and parse IP, TCP and UDP.

EXP 14

AIM

To write and implement a network packet sniffer using Raw Sockets in C to capture and parse IP, TCP, and UDP network headers.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Open a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_TCP (or ETH_P_ALL for lower-level link layer capture).
2. **Buffer Allocation:** Allocate a buffer (e.g., 65536 bytes) to hold incoming network frames and packets.
3. **Capture Loop:**
   - Call recvfrom() to continuously capture network packets passing through the interface.
4. **Parse Headers:**
   - **Extract IP Header:** Cast the buffer pointer to an struct iphdr pointer. Extract fields like Version, Header Length, TTL, Protocol, Source IP, and Destination IP.
   - **Identify Protocol:** Check the protocol field (ip_hdr->protocol).
   - **Extract Transport Header:**
     - If Protocol is 6 (TCP): Cast buffer offset to struct tcphdr and extract Source Port, Destination Port, and Flags (SYN, ACK, FIN, etc.).

     If Protocol is 17 (UDP): Cast buffer offset to struct udphdr and extract Source Port, Destination Port, and Length.
5. **Display Packet Info:** Print formatted packet metadata to the console.
6. **Clean Up:** Close the socket upon receiving a termination signal.

Code Implementation (packet_sniffer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 65536

void process_packet(unsigned char *buffer, int size) {
  // Extract IP Header
  struct iphdr *ip_hdr = (struct iphdr *)buffer;
  unsigned int ip_hdr_len = ip_hdr->ihl * 4;

  struct sockaddr_in source, dest;
  memset(&source, 0, sizeof(source));
  source.sin_addr.s_addr = ip_hdr->saddr;

  memset(&dest, 0, sizeof(dest));
  dest.sin_addr.s_addr = ip_hdr->daddr;

  printf("\n================ Packet Captured
================\n");
```

Result :

```
    printf("IP Header:\n");
    printf("   |-IP Version        : %d\n", (unsigned int)ip_hdr->version);
    printf("   |-Header Length     : %d Bytes\n", ip_hdr_len);
    printf("   |-Time To Live (TTL): %d\n", ip_hdr->ttl);
    printf("   |-Protocol          : %d\n", ip_hdr->protocol);
    printf("   |-Source IP         : %s\n", inet_ntoa(source.sin_addr));
    printf("   |-Destination IP    : %s\n", inet_ntoa(dest.sin_addr));

    // Handle TCP Packets
    if (ip_hdr->protocol == IPPROTO_TCP) {
        struct tcphdr *tcp_hdr = (struct tcphdr *)(buffer + ip_hdr_len);
        printf("TCP Header:\n");
        printf("   |-Source Port       : %u\n", ntohs(tcp_hdr->source));
        printf("   |-Destination Port  : %u\n", ntohs(tcp_hdr->dest));
        printf("   |-Sequence Number   : %u\n", ntohl(tcp_hdr->seq));
        printf("   |-Acknowledge No    : %u\n", ntohl(tcp_hdr->ack_seq));
        printf("   |-SYN Flag          : %d\n", tcp_hdr->syn);
        printf("   |-ACK Flag          : %d\n", tcp_hdr->ack);
        printf("   |-FIN Flag          : %d\n", tcp_hdr->fin);
    }
    // Handle UDP Packets
    else if (ip_hdr->protocol == IPPROTO_UDP) {
        struct udphdr *udp_hdr = (struct udphdr *)(buffer + ip_hdr_len);
        printf("UDP Header:\n");
        printf("   |-Source Port       : %u\n", ntohs(udp_hdr->source));
        printf("   |-Destination Port  : %u\n", ntohs(udp_hdr->dest));
        printf("   |-UDP Length        : %u\n", ntohs(udp_hdr->len));
    }

    printf("=============================================
    =\n");
}

int main() {
    int raw_sock;
    socklen_t saddr_len;
    struct sockaddr saddr;
    unsigned char *buffer = (unsigned char *)malloc(BUFFER_SIZE);

    if (buffer == NULL) {
        printf("Buffer allocation failed.\n");
        return 1;
    }

    // Create raw socket to capture IP packets
    raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (raw_sock < 0) {
        perror("Socket creation error (ensure you run with sudo)");
        free(buffer);
        return 1;
    }

    printf("Starting Packet Sniffer...\nPress Ctrl+C to stop.\n");
```

```
    // Capture 3 packets as an example output
    int packet_count = 0;
    while (packet_count < 3) {
      saddr_len = sizeof(saddr);
      int data_size = recvfrom(raw_sock, buffer, BUFFER_SIZE, 0,
&saddr, &saddr_len);

      if (data_size < 0) {
        perror("Recvfrom error");
        free(buffer);
        close(raw_sock);
        return 1;
      }

      process_packet(buffer, data_size);
      packet_count++;
    }

    close(raw_sock);
    free(buffer);
    return 0;
}
```

Input / Output

Input

Command Terminal:

```
sudo ./sniffer
```

Output

Terminal Screen:

```
Starting Packet Sniffer...
Press Ctrl+C to stop.

================ Packet Captured ================
IP Header:
   |-IP Version        : 4
   |-Header Length     : 20 Bytes
   |-Time To Live (TTL): 64
   |-Protocol          : 6
   |-Source IP         : 192.168.1.5
   |-Destination IP    : 93.184.216.34
TCP Header:
   |-Source Port       : 49152
   |-Destination Port  : 80
   |-Sequence Number   : 10423011
   |-Acknowledge No    : 0
   |-SYN Flag          : 1
   |-ACK Flag          : 0
   |-FIN Flag          : 0
================================================
```

Exp: NO: 15
Date: 5/10/26

Types of web logs using webalizer log Analysis.

Aim:

To analyze different types of web logs using the webalizer log analysis tool.

Result:

Thus different types of web logs using the webalizer log analysis is analyzed successfully.

EXP 15

AIM

To analyze different types of web logs using the Webalizer log analysis tool and generate web traffic reports.

Procedure

Step 1: Run Webalizer

1. Download and extract the Webalizer executable (webalizer.exe) on Windows.
2. Place the executable in a dedicated working folder (e.g., C:\webalizer).

Step 2: Input Web Log File

1. Obtain or download a sample web server access log file (e.g., access.log from Apache, Nginx, or IIS).
2. Move the log file into the same directory as webalizer.exe or update the webalizer.conf configuration file to point to the input log location:

```
LogFile access.log
OutputDir ./output
```

Step 3: Run Webalizer

1. Open Command Prompt (cmd) and navigate to the Webalizer directory:

```
DOS
cd C:\webalizer
```

2. Execute Webalizer specifying the log file:

```
DOS
webalizer.exe access.log
```

3. Open the generated index.html file in any web browser to view the generated traffic report.

Choose Logfiles | Logfile | View | Settings | Additional HTML Code | Hide / Group / Ignore / Include | Format Graphs | Configurationfile

Input:

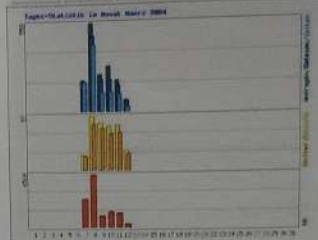
Logfiles:

C:\Users\TCS\Downloads\access_log

Target Directory:

C:\Users\TCS\

Monthly statistics



Tages Statistik im Monat Marz 2004

Tag	Anfragen Summe	Anfragen Schnitt	Anfragen Maximum	Dateien Summe	Dateien Schnitt	Dateien Maximum	Seiten Summe	Seiten Schnitt	Seiten Maximum	Besuche Summe	Besuche Schnitt	Besuche Maximum	Rechner Summe	Rechner Schnitt	Rechner Maximum	kb Summe	kb Schnitt	kb Maximum
7	197 12.74%	28	29	147 11.54%	18	21	190 11.94%	23	27	15 6.41%	2	3	14 8.19%	2	3	2183 22.27%	298	542
8	559 36.16%	24	63	478 37.52%	21	56	419 26.47%	18	40	57 24.38%	2	7	48 28.07%	2	7	4569 46.54%	196	886
9	232 15.01%	11	29	205 16.09%	10	29	318 20.02%	16	38	49 20.94%	2	5	44 25.73%	2	4	955 9.73%	45	130
10	266 18.50%	11	102	205 16.09%	10	33	324 20.47%	16	28	46 19.66%	2	5	30 22.22%	2	4	1116 22.68%	47	370

Hosts



Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net
2	72 4.66%	71 5.57%	52 5.30%	86 0.80%	71 30.34%	0.09	6.62	inktomisearch.com
3	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
4	44 2.85%	42 3.30%	23 2.34%	244 2.28%	2 0.85%	4.51	4.70	hevanet.com
5	35 2.26%	29 2.28%	14 1.43%	218 2.03%	7 2.99%	1.31	6.00	bc.ca
6	29 1.88%	28 2.20%	14 1.43%	97 0.91%	1 0.43%	3.43	3.43	panduit.com
7	23 1.49%	14 1.10%	10 1.02%	135 1.26%	1 0.43%	12.35	12.35	geovariances.fr
8	22 1.42%	22 1.73%	8 0.81%	67 0.63%	1 0.43%	3.72	3.72	cox.net
9	19 1.23%	19 1.49%	7 0.71%	61 0.57%	1 0.43%	1.52	1.52	ac.il
10	19 1.23%	11 0.86%	10 1.02%	51 0.48%	2 0.85%	3.04	5.87	netinfo.bg
11	15 0.97%	14 1.10%	13 1.32%	130 1.22%	3 1.28%	1.29	3.87	e-i.net
12	13 0.84%	13 1.02%	13 1.32%	120 1.12%	1 0.43%	3.88	3.88	telia.net
13	13 0.84%	13 1.02%	1 0.10%	28 0.26%	1 0.43%	0.13	0.13	net.ar
14	13 0.84%	13 1.02%	2 0.20%	30 0.28%	1 0.43%	0.87	0.87	dhl.com
15	12 0.78%	11 0.86%	9 0.92%	68 0.63%	1 0.43%	25.53	25.53	tiscali.de
16	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.33	0.33	wtbts.org
17	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.10	0.10	qwest.net
18	11 0.71%	11 0.86%	7 0.71%	41 0.38%	2 0.85%	0.56	1.03	radiant.net
19	10 0.65%	9 0.71%	7 0.71%	41 0.38%	1 0.43%	1.40	1.40	net.au
20	10 0.65%	10 0.78%	2 0.20%	61 0.57%	1 0.43%	0.17	0.17	3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
2	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net

User-agents

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries

9 72 4.66% 71 5.57% 52 5.30% 86 0.80% 71 30.34% 0.09 6.62 **inktomisearch.com**

10 4 0.26% 4 0.31% 4 0.41% 74 0.69% 4 1.71% 0.00 0.00 **rhyolite.com**

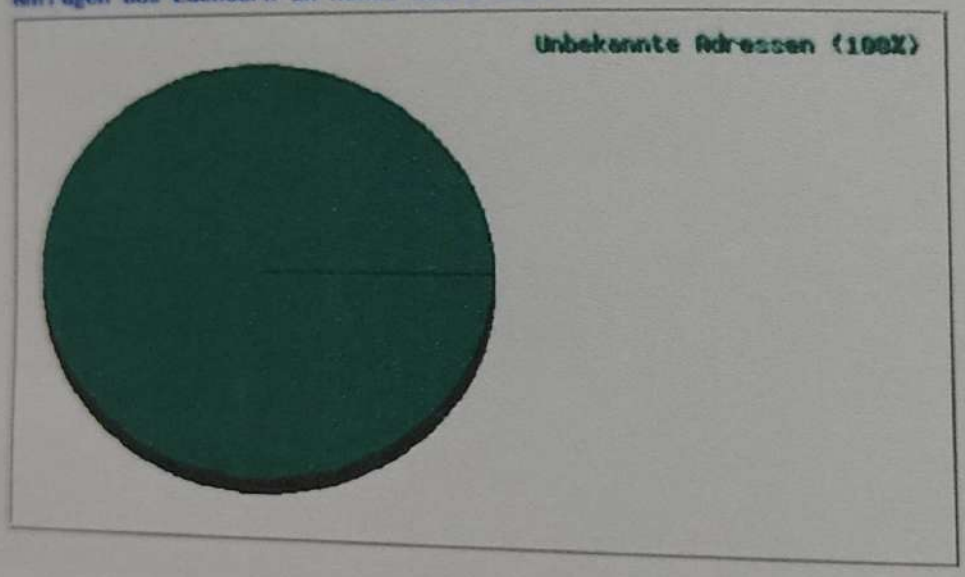
Top 1 von 1 Verweise

#	Anfragen		Besuche		Verweise
1	1546	100.00%	234	100.00%	- (Direkte Anfrage)

Top 1 von 1 Anwenderprogramme

#	Anfragen		kb		Besuche		Anwenderprogramm
1	1546	100.00%	10699	100.00%	234	100.00%	-

Anfragen aus Laendern im Monat Maerz 2004



Top 1 von 1 Ländern

#	Anfragen		Dateien		kb		Besuche		Land
1	1543	99.81%	1271	99.76%	10684	99.85%	231	100.00%	Unbekannte Adressen

100% MicroSoft free!

Completed